



Denis Kuramshin <denis911@gmail.com>

AI Consilium - proceed with SDD - LLM zoomcamp capstone 1

2 messages

Denis Kuramshin <denis911@gmail.com>
To: Denis Kuramshin <denis911@gmail.com>

Sat, Jul 25, 2026 at 10:51 AM

AI Consilium - proceed with SDD as described in Spec-Driven Development, like in this article:

<https://alexeyondata.substack.com/p/ai-native-development-specifications>

or here

<https://www.deeplearning.ai/courses/spec-driven-development-with-coding-agents>

===

To successfully adopt **Spec-Driven Development (SDD)** as a solo developer, you are essentially adopting an **"Editor-in-Chief"** mindset. You will transition from typing out raw syntax to defining constraints, reviewing specifications, and validating autonomous pull requests.

Using **Antigravity 2.0 (Desktop Team Manager)**, **Antigravity IDE (Interactive Code Editor)**, and **Google Labs Jules (Repository Worker)**, your end-to-end SDD workflow for **AI Consilium** will operate across three high-level phases.

Phase 1: High-Level Planning & Spec Drafting (Surface: Antigravity 2.0)

*This is your desktop command center where you act as the **Product Manager + Lead Architect**.*

1. **The Brainstorm:** You open Antigravity 2.0 and spark a conversation with your local agent team (e.g., an Orchestrator, a Product Manager agent, and an Architect agent). You pitch the vision: *"I want to build AI Consilium. It needs to query 5 LLMs via Python, summarize the consensus, generate a Mermaid.js Gantt chart if relevant, and save it directly to my Obsidian vault."*
2. **Deconstruction:** The Antigravity 2.0 agents cross-reference your architecture rules (e.g., using FastAPI, Python async, sentence-case markdown headers). They break this massive goal down into a logical roadmap of micro-features.
3. **Spec Compilation:** For each micro-feature, the agents generate a highly rigorous markdown specification (following the exact SDD philosophies from Alexey and DeepLearning.AI).
4. **Publishing:** Antigravity 2.0 uses your local GitHub CLI (`gh`) to automatically push these specifications directly onto your `ai-consilium` GitHub repository as structured **GitHub Issues**.
 - *Crucial Step:* The first line of each issue description automatically reads: `@google-jules`.

Phase 2: Asynchronous Execution (Surface: GitHub & Google Labs Jules)

This is the "Fire & Forget" phase. You can literally close your laptop or go eat lunch while the work happens.

1. **Jules Wakes Up:** The moment Antigravity 2.0 creates the issue, a GitHub Webhook triggers the **Jules GitHub App**.
2. **Sandbox Provisioning:** In the Google Cloud backend, Jules spins up a secure, isolated Virtual Machine. It automatically clones your repository, reads the system configuration, and analyzes the environment.
3. **The Engineering Loop:** Jules starts implementing the spec. It writes `providers.py`, sets up the async orchestration, and constructs the Obsidian markdown exporter.
4. **The Adversarial Critic:** Jules finishes writing the code but doesn't hand it to you yet. It runs a local test suite inside the VM. If a test fails, or if its internal "Critic" model finds an edge case (like handling an API timeout from Grok), Jules modifies its own code and re-runs the tests until it passes.
5. **The Pull Request:** Jules pushes a clean branch to GitHub and opens a **Pull Request (PR)**, mapping it back to the original issue.

Phase 3: Review, Refinement, & Land (Surface: Antigravity IDE)

This is where you step back in as the **Senior Reviewer** to merge the code.

1. **Pulling the Branch:** You open your **Antigravity IDE**. You see the notifications that Jules has submitted 2 or 3 Pull Requests for different parts of the project.
2. **Code Review:** You fetch Jules' branch into your IDE. You don't have to guess what it did—Jules includes a complete execution log and a summary of structural changes in the PR.
3. **Interactive Tweaks:** If you notice a styling preference you forgot to include in the original specification (e.g., *"I want the Obsidian folders to use a different naming convention"*), you don't need to manually rewrite it. You simply open the file, use the **Antigravity IDE inline assistant**, and type: *"Quickly adjust this path routing to match my local vault layout."*
4. **The Merge:** You run the final local check, press merge on GitHub, and the first working module of **AI Consilium** is officially part of your main codebase.

How to Start Right Now (Your First Actionable Step)

To build your very first specification without getting overwhelmed, let's start at **Phase 1**.

Open **Antigravity 2.0** (or let's simulate it right here), and prompt the planning agent with this:

"Act as my Product Manager and Principal Architect. We are building the first component of 'AI Consilium': a robust, asynchronous Python module to hit the free tier APIs of OpenAI, Anthropic, and Gemini concurrently. Review our architecture needs and write the exact SDD GitHub Issue markdown text—including the @google-jules trigger—so I can push it to my repository."

Would you like me to act as your **Antigravity 2.0 agent** right now and generate that exact first specification markdown for you to copy into your repository?

===

(Added requirement to keep API-keys-only access to LLM providers)

This is a massive improvement to the architecture. Relying on **API keys** via an Open-AI compatible framework or native SDKs is exponentially cleaner, safer, and faster than brittle headless browser scraping (which breaks every time a provider updates their UI CSS classes).

By structuring the code to use **Asynchronous HTTP requests** wrapped inside a `.env` configuration file, you completely safeguard your operational credentials while letting Jules run tests natively in its cloud sandbox via your `.env.example` mapping.

Below is your official **Antigravity 2.0 Spec-Driven Development Blueprint** formatted exactly as a GitHub Issue for you to copy and paste to initialize **Jules**.

Copy-Paste into a New GitHub Issue:

@google-jules

Overview

Implement Phase 1 of 'AI Consilium'—an asynchronous Python engine that concurrently requests identical research prompts from five frontier LLM providers (OpenAI, Anthropic, Gemini, Perplexity, and Grok) using natively authorized API keys. The configurations must be safely pulled from a local environment variable matrix.

Core Architectural Requirements

1. ****Environment & Security Sandbox:****
 - Implement configuration management using ``python-dotenv``.

- Never hardcode or commit keys. Read directly from the host system's environment.
- Create a boilerplate file named `.env.example` showing exactly what variable names are needed.

2. **Multi-Provider Asynchronous Query Engine (`council/providers.py`):**

- Implement concurrent execution using `asyncio` and `httpx` (or native SDK async clients) to blast the prompt to all 5 endpoints at the exact same time.
- Fallback and isolate failed requests: If Grok or Perplexity times out or errors, catch the exception locally, log the failure, and return data from the remaining successful providers without failing the entire script.

3. **Target Providers & Model Schemas:**

- **ChatGPT:** Call the `chat/completions` endpoint utilizing the `gpt-4o-mini` schema via `OPENAI_API_KEY`.
- **Claude:** Call the Anthropic Messages API utilizing the `claude-3-5-haiku` schema via `ANTHROPIC_API_KEY`.
- **Gemini:** Call the Gemini Developer API via Google AI Studio utilizing the `gemini-2.5-flash` schema via `GEMINI_API_KEY`.
- **Perplexity:** Call the Sonar API (`api.perplexity.ai`) utilizing the `sonar` standard base model via `PERPLEXITY_API_KEY`.
- **Grok:** Call the xAI API console (`api.x.ai`) utilizing the `grok-4.1-fast` schema via `XAI_API_KEY`.

4. **Response Aggregation Output:**

- Create `main.py` as the command-line CLI entrypoint.
- Running `python main.py --query "Your Question Here"` should execute the pipeline and output a structured JSON schema containing:

```
```json
{
 "query": "Your Question Here",
 "timestamp": "ISO-8601",
 "responses": {
 "openai": "...",
 "anthropic": "...",
 "gemini": "...",
 "perplexity": "...",
 "grok": "..."
 }
}
```
```

Directory Structure to Generate

```
```text
ai-consilium/
├── .env.example
├── .gitignore
├── requirements.txt
├── main.py
├── council/
│ ├── __init__.py
│ └── providers.py
...
```
```

Definition of Done (DoD)

- Running `pytest` validates the async initialization of all five provider stubs.
- `.gitignore` explicitly blocks any file matching `.env`.
- Empty strings or mock fallbacks are properly mapped if an API key is missing or invalid during runtime.

Your Local Sandbox Files

To ensure that Jules has a baseline workspace to operate on and run its tests successfully, create these two simple files locally using your **Antigravity IDE** before committing and pushing the issue:

1. `.env.example` (Commit to GitHub)

```
# AI Consilium Local Environment Config Example
# Copy this file to '.env' on your local system and fill in your keys.

OPENAI_API_KEY=your_openai_api_key_here
ANTHROPIC_API_KEY=your_anthropic_api_key_here
GEMINI_API_KEY=your_google_aistudio_key_here
PERPLEXITY_API_KEY=your_pplx_api_key_here
XAI_API_KEY=your_grok_xai_api_key_here
```

2. `.gitignore` (Commit to GitHub)

```
# Python bytecode
__pycache__/
*.py[.cod]
*$py.class

# Environments
.env
.venv
env/
venv/
ENV/
```

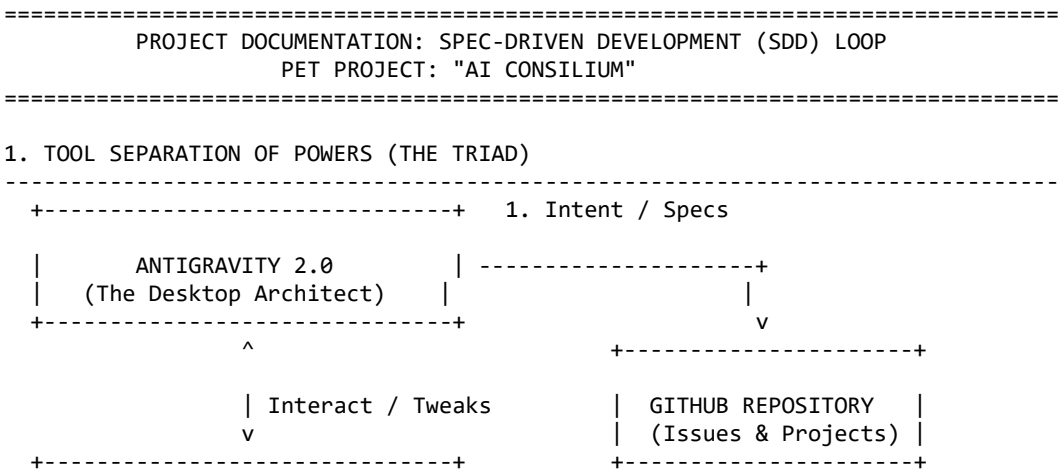
 What Happens Next in Your Workflow?

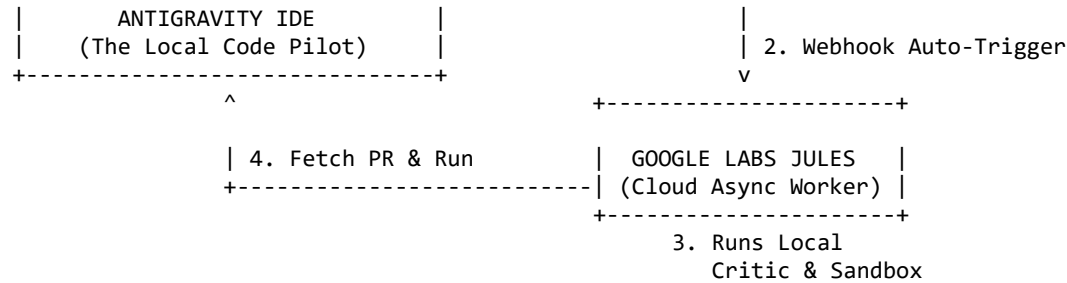
- 1. Commit `.env.example` and `.gitignore` to your repo, and push them to your `main` branch.
- 2. Open the GitHub Issue using the markdown text block provided above.
- 3. **Google Labs Jules** will capture the issue, boot up a clean cloud container, read your repository file layout, and intelligently map the logic inside `council/providers.py`.
- 4. Once Jules outputs the Pull Request, you will fetch its branch directly into your **Antigravity IDE** to run it locally with your real `.env` file containing your active API keys!

===

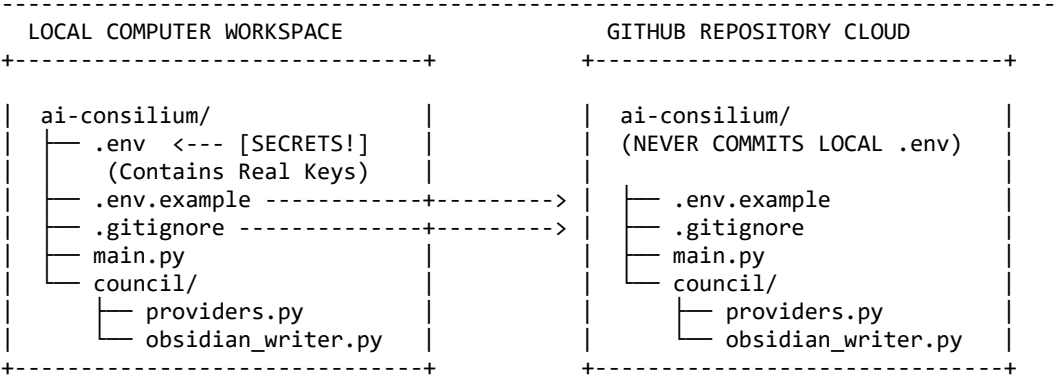
Summary:

Spec-Driven Development (SDD) for the "AI Consilium" project utilizes a tri-level architecture, dividing tasks between local design (Antigravity 2.0), cloud-based asynchronous development (Google Labs Jules), and local IDE validation (Antigravity IDE). The workflow involves generating markdown specifications locally, creating GitHub issues tagged for Jules to trigger automated cloud development, and validating the resulting code locally using a secured `.env` configuration file to separate secret keys from the repository.





2. PROJECT SANDBOX WORKSPACE MAP



3. THE END-TO-END AUTOMATION PIPELINE (STEP-BY-STEP)

[DESIGN] -> [SPECIFICATION] -> [PROVISIONING] -> [EXECUTION] -> [INTEGRATION]

To begin, initialize a GitHub repository, activate the Jules app, and utilize the following template for new feature specifications:

```
@google-jules

## Overview
[Description]

## Directory Structure
[Structure]

## Definition of Done (DoD)
- [ ] Tests pass

===
```

Denis Kuramshin <denis911@gmail.com>

Sat, Jul 25, 2026 at 1:27 PM

To: Denis Kuramshin <denis911@gmail.com>

Adding Linear -

The debate between **GitHub Issues** and **Linear Tickets** represents a fundamental choice in how software development is managed, especially when transitioning to an **AI-Native, Spec-Driven Development (SDD)** model. [1, 2]

When analyzing how these interfaces function—particularly in the context of project management paradigms highlighted in industry workflows—the differences become stark, especially when using an autonomous cloud agent like **Jules**. [2, 3, 4]

1. The Core Philosophical Difference

- **GitHub Issues (Code-First Collaboration):**
GitHub Issues exist natively directly next to your code branches, commits, and pull requests. It is optimized for **visibility and execution**. There is no platform-hopping; an issue can naturally transition into a Pull Request. It treats the codebase as the ultimate source of truth, making it a highly pragmatic workspace for solo developers or fast-moving engineering teams. [3, 5, 6, 7]
- **Linear Tickets (Velocity-First Product Management):**
Linear is built specifically to move fast with an ultra-responsive, local-first UI engine that operates sub-100ms. It is heavily optimized for **triage, priority mapping, and product management**. Linear treats the development cycle as a sequence of tightly structured decisions. It handles backlogs, cycle analytics, and cross-project tracking with extreme polish. [2, 3, 8]

2. How Both Fit into Spec-Driven Development (SDD)

When implementing an agentic SDD workflow for a solo setup (like **AI Consilium**), your ticketing system changes from a mere tracking board into a **prompt payload repository**. [2]

Option A: The Pure GitHub Issues Approach

If you stick exclusively to GitHub Issues, your loop remains close to the repository metal. [5, 6]

- **The Workflow:** You use **Antigravity 2.0** to draft your functional specification. It executes a local script (`gh issue create`) to post that text directly as an issue.
- **How Jules Sees It:** Jules reads the markdown payload straight from the repository interface natively. It writes the branch, executes the changes in its isolated Google Cloud VM, maps it directly back, and resolves the issue automatically via standard GitHub linkages. [5, 9, 10]
- **The Verdict:** Simple, zero architectural overhead, completely free, but lacks complex tracking or deep visual triage metrics. [3]

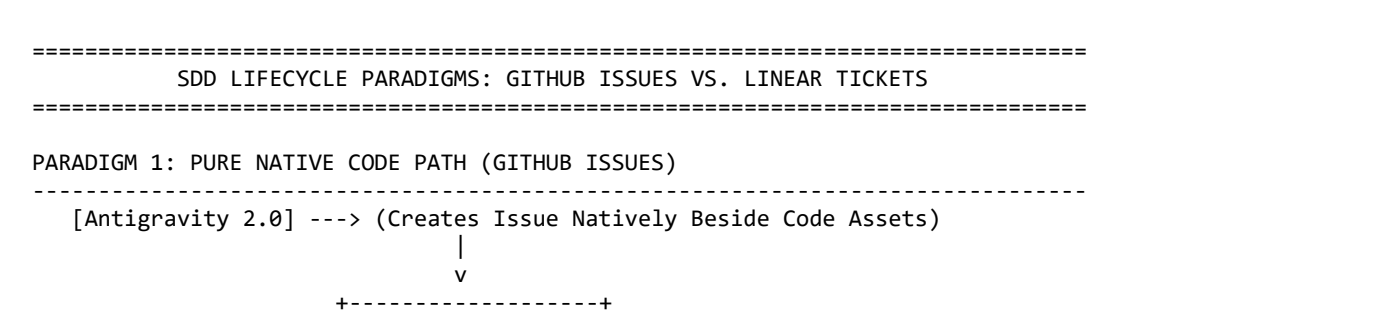
Option B: The Linear Integration Approach

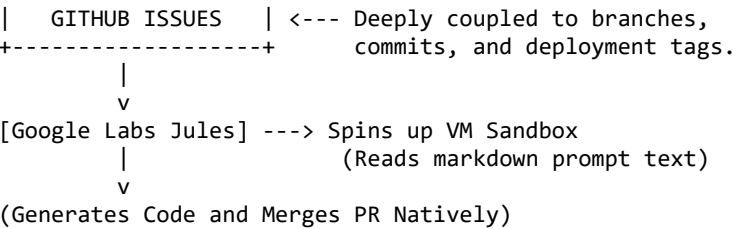
Because Linear embraces AI-driven workflows natively, it supports **Model Context Protocol (MCP)** server interactions directly. In fact, **Google Labs Jules natively connects to the Linear MCP server**. [2, 4, 11]

- **The Workflow:** Your **Antigravity 2.0** or your local AI project-manager agent talks directly to the Linear API. It structures the specification, sets first-class priority tokens (e.g., Urgent/High), and places it cleanly inside an active sprint loop. [2, 6, 8, 12]
- **How Jules Sees It:** Because of its **Linear MCP integration**, you can provide Jules with a token to your Linear workspace. Jules can use automated tools to scan your active Linear tickets, pull the specification body as context, cross-reference it against your repository schema, write the branch on GitHub, and cross-post updates directly back to Linear when the work is done. [11, 12, 13]
- **The Verdict:** Fast, high-level structural control over your task lifecycle, but requires authenticating an external tool layer with MCP connections. [11, 14]

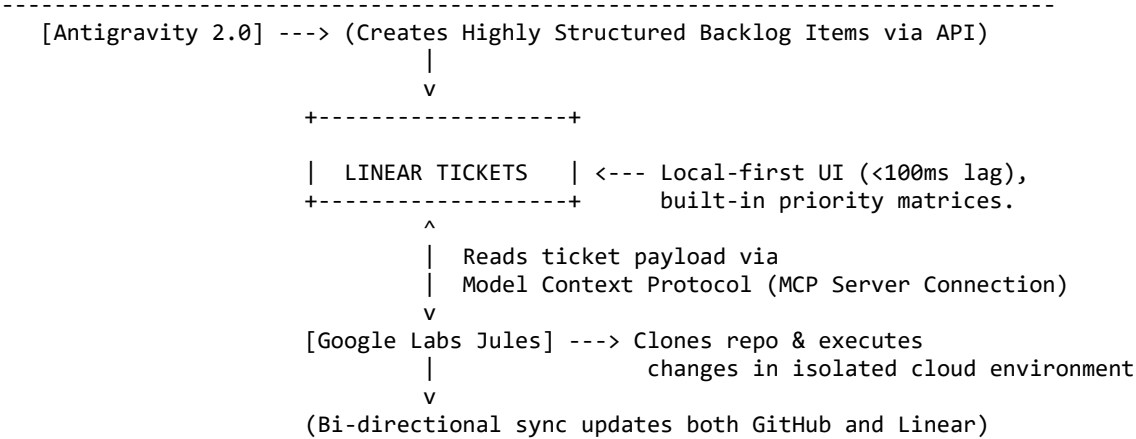
3. Comprehensive Summary Diagram (ASCII Only)

Below is an updated system-level visualization detailing exactly how the architecture adapts depending on whether you choose a native Git-centric or product-centric tracking engine.





PARADIGM 2: HIGH-VELOCITY PRODUCT MANAGEMENT PATH (LINEAR TICKETS)



| COMPARATIVE METRIC GRID FOR SOLO AGENTIC BUILDERS | | |
|---|--------------------------------|-----------------------------------|
| FEATURE | GITHUB ISSUES | LINEAR TICKETS |
| Setup Friction | Zero (Native to your Git repo) | Low (Requires MCP Setup/Token) |
| Interface Speed | Standard Server Roundtrips | Instant (Local Cache Sync Engine) |
| AI Agent Integration | Built-in Webhook Triggers | Deep MCP Platform Capabilities |
| Cost Matrix | 100% Free for Private Repos | Generous Free Tier for Solo |

How to Proceed on "AI Consilium"

Since you are starting out and learning the ropes, the recommended strategy is to **start with Paradigm 1 (GitHub Issues)** to eliminate platform friction. Once your loop between Antigravity and Jules is flawless, you can effortlessly swap out the issue script for a **Linear MCP pipeline** to manage your multi-tenant startup priorities. [2, 4]

If you would like to map out the **Linear configuration block** for Jules' settings later on, let me know:

- Would you prefer to view the raw **GraphQL configuration syntax** for an external tool script?
- Or should we focus first on running a test validation suite for the **AI Consilium** python code we generated? [10]

[1] <https://intent-driven.dev>
[2] <https://octopusbuilds.com>
[3] <https://blog.vibecoder.me>
[4] <https://jules.google>
[5] <https://medium.com>
[6] <https://www.devtoolreviews.com>
[7] <https://www.youtube.com>
[8] <https://coteria.co>
[9] <https://jules.google>
[10] <https://www.youtube.com>
[11] <https://linear.app>
[12] <https://www.adopt.ai>
[13] <https://github.com>

25/07/2026, 20:23

Gmail - AI Consilium - proceed with SDD - LLM zoomcamp capstone 1

[14] <https://www.adopt.ai>

===

[Quoted text hidden]